Asistente Educativo UNIANDES

Orientación sobre titulación y normativa académica, fundamentada en documentos verificables

¿Qué busca el proyecto?



Ayudar a estudiantes a comprender trámites y normativa.



Organizar y documentar fuentes oficiales.



Responder con artículos y secciones concretas.



Facilitar la revisión humana de cada orientación.

¿Cómo busca ser fiable?

Fuentes aprobadas y vigentes.

↓

RAG: recuperar fragmentos pertinentes.

↓

Generar orientación con evidencia.

↓

Validar IDs citados y revisión de la fuente.

↓

Mostrar documento, artículo/sección y página.

Sin evidencia suficiente: abstenerse y derivar a Secretaría.

Citas trazables ≠ garantía de exactitud: verificar contenido y aplicabilidad.

Ejemplo de cita (estructura): Documento → Artículo / sección → Página → Revisión

Tecnologías y herramientas empleadas

- **Interfaz:** React · TypeScript · Vite.
- **Backend:** Python · FastAPI · Pydantic · Uvicorn.
- **Conocimiento:** SQLite · FTS5/BM25 · vectores · RRF.
- **Modelos de respuesta:** Gemini (gemini-3.5-flash-lite) · OpenAI (gpt-4.1-mini), con respaldo.
- **Embeddings, uno elegido:** gemini-embedding-001 o text-embedding-3-small.
- **Desarrollo y revisión:** Git · GitHub · asistencia de Codex.
- **Pruebas:** pytest/cobertura · Vitest · Playwright · Ruff · Gitleaks.
- **Despliegue:** Docker · GitHub Actions · Cloud Run · Vercel · Secret Manager / WIF.

Herramientas del asistente

Buscar fragmentos. | **Consultar requisitos.** | **Generar checklist orientativo.** | **Simular días hábiles.** | **Preparar borrador demo** con confirmación.

El backend valida y ejecuta las herramientas; el modelo puede solicitarlas.

Proyecto educativo: no emite resoluciones, no certifica cumplimiento ni modifica expedientes. Confirmar decisiones con los canales institucionales.

Estructura del proyecto y ejecución

Asistente Educativo UNIANDES · React / Vite + FastAPI

Directorios del proyecto (selectivo)

SeminarioComplexivoGrupo1/

- **frontend/**
  - src/main.tsx → entrada y rutas
  - src/App.tsx → chat
  - src/api.ts → cliente HTTP / SSE
  - **src/admin/** → panel administrativo
  - **src/Metricas.tsx** → dashboard
  - **e2e/** → pruebas de navegador
  - package.json · vite.config.ts · vercel.json
- **backend/**
  - app/main.py → API FastAPI
  - app/config.py · schemas.py → configuración y contratos
  - app/flujo.py · agente.py → orquestación
  - **app/services/** → rag, llm, chat_flow, actions, metrics, analysis
  - app/tools/academicas.py → herramientas
  - app/admin_routes.py · admin_auth.py → administración
  - app/prompts/system_v*.md → prompts
  - **tests/** · requirements-dev.txt · **Dockerfile**

- **config/** → fuentes y parámetros
- **data/raw/** → originales y OCR
- **scripts/** → ingesta, smoke y verificadores
- **eval/** → casos y evaluación
- **docs/** → guías, PDF y evidencias
- **.github/workflows/** → CI/CD

Árbol resumido: los tests unitarios del frontend también viven en src/.

Arquitectura (vista general)

Navegador · React / TypeScript

↓

API FastAPI · Python / Uvicorn

↓

Flujo + RAG + agente

↓

SQLite / FTS5 / vectores · **Gemini / OpenAI**

Desarrollo: Vite :5173 y API :8000

Producción: Vercel frontend + proxy /api → Cloud Run backend

Las claves se leen solo en backend; no se envían al navegador.

En desarrollo se conecta a la API local (:8000) y se usa el proxy administrativo del frontend (p. ej. /api) para rutas de administración.

Inicio local · Python 3.12 y Node ≥22

Todos los comandos se ejecutan desde la raíz salvo bloques que indican cd.

A Preparar backend (desde raíz)

```
python -m venv backend/.venv
source backend/.venv/bin/activate
```

Windows PowerShell: `backend\\.venv\\Scripts\\Activate.ps1`

```
python -m pip install -r backend/requirements-dev.txt
python scripts/ingest.py --strategy lexical
```

B Terminal 1 · API

```
cd backend
python -m uvicorn app.main:app --reload --port 8000
```

C Terminal 2 · Web

```
cd frentend
npm ci
npm run dev
```

Chat: localhost:5173 · **Admin:** /admin · **Métricas:** /metricas

API: 127.0.0.1:8000 · **Contrato:** /docs · **Salud:** /health

Verificación

Desde raíz · backend

```
python -m pytest -q -c backend/pyproject.toml backend/tests --cov=app --cov-fail-under=85
```

Desde frontend/

```
npm test
npm run build
npm run test:admin
```

Playwright requiere Chromium; las pruebas usan API aislada y modelo doble.

La ingesta léxica no necesita API de modelos. Para generación o embeddings híbridos, configurar claves privadas en backend. **No sobrescribir un .env existente ni incluirlo en Git.**

Prompts y documentos: carga, versiones y aprobación

Panel administrativo del Asistente Educativo UNIANDES

Versionado del panel y auditoría; distinto de los commits Git.

1. Gestión de prompts

Versión identifica un prompt; revisión registra sus ediciones.

expected_revision: evita sobrescribir cambios ajenos; conflicto → recargar.

Administrador inicia sesión → Crear prompt: nueva versión V1, V2, V3... → Editar contenido y descripción → Guardar archivo .md + registro y auditoría → Seleccionar active_prompt en configuración → Prompt activo para el chat

Editar contenido y descripción → Comparar versiones: mismos casos y evidencia, proveedor elegido, sin fallback → Resultados y trazas para revisión humana

Los prompts se seleccionan y editan; no tienen un estado approved propio.

Comparar no activa automáticamente un ganador.

2. Gestión de documentos

Upload: archivo + metadatos → Validar formato, tamaño y texto; registrar SHA-256 → Guardar original + revisión y auditoría → Revisión humana: contenido, vigencia y aplicabilidad → Aprobar y habilitar la fuente → Indexar la revisión → Fragmentos con ID, sección y página; FTS5 y vectores opcionales

Metadatos: Título · tipo · URL · fechas · approved · enabled

expected_revision: control de concurrencia.

Guardar original + revisión y auditoría → Reemplazar archivo → Nueva revisión; approved=false obligatorio → Revisión humana: contenido, vigencia y aplicabilidad

Sin aprobación, deshabilitado, fuera de vigencia o índice desactualizado → no se usa como evidencia.

Indexar no equivale a aprobar; aprobar no sustituye indexar.

3. Uso en una consulta

Prompt activo + fragmentos elegibles → Gemini / OpenAI: orientación → Validar JSON, IDs y revisión de citas → Respuesta con documento, artículo/sección, página y revisión

En modo híbrido, indexar puede llamar al modelo de embeddings; en léxico no.

Controles del sistema

- **Acceso administrativo:** sesión HttpOnly + guard de mutaciones.
- **Auditoría:** quién cambió qué y revisiones anteriores.
- **Cambiar estrategia de** embeddings o fragmentación puede exigir reindexar.
- **SQLite y archivos locales:** Cloud Run no garantiza conservar cambios entre instancias.

Revisión humana de fuentes ≠ aprobación automática del modelo. Orientación educativa; verificar con canales institucionales.

UNIANDES | Asistente Académico UNIANDES

Nuestro proceso RAG

1. Preparar el conocimiento

Documentos oficiales PDF + OCR

Verificar SHA-256 y metadatos

Revisión humana: aprobación, habilitación y vigencia

Fragmentar con página, sección e ID

(1400 caracteres · solapamiento 160)

Elegir **UNO:** gemini-embedding-001 / text-embedding-3-small · caché

Opcional

Embeddings de documentos

SQLite

Texto + FTS5

Vectores de 768 dimensiones (solo híbrido)

2. Responder una consulta

Pregunta + contexto académico

Validación y filtro de inyección

Rechazo local, sin modelo

Recuperar fuentes elegibles

Léxico: FTS5 / BM25 · sin API

Semántico opcional: embedding de pregunta con el **MISMO** modelo del índice → coseno

Fusionar rankings RRF → top-k = 4

¿Hay evidencia suficiente?

NO → Abstención sin generación

SÍ → Prompt + pregunta + evidencia → Gemini / OpenAI → Validar JSON, IDs citados y vigencia → Respuesta con citas: documento, sección, página y revisión

El agente puede solicitar herramientas validadas; máximo 4 vueltas.

Orientación educativa; verificar con Secretaría.

Notas importantes:

- La semilla Docker usa modo léxico.
- Chat y embeddings usan modelos distintos; elegir OpenAI en chat no cambia el índice.
- Las citas estructurales no garantizan respaldo semántico de cada afirmación.

Nuestro flujo de desarrollo y CI/CD

Asistente Educativo UNIANDES · GitHub → Vercel + Google Cloud Run

1 Desarrollo y revisión con Git

Actualizar main: `git pull` → Crear rama de trabajo → Implementar código + pruebas + documentación → Validar localmente → Commit + push → Pull Request y revisión → Preview del frontend en Vercel → Checks aprobados y revisión del equipo → Merge a main

El merge lo confirma el equipo, no es un proceso automático de aprobación.

2 Integración continua · GitHub Actions

Push / PR

- **Secretos:** Gitleaks + detector de constantes. Los escáneres tienen alcance; revisión histórica adicional.
- **Backend:** pytest + cobertura ≥85 % + Ruff + evaluación sin modelos + auditoría.
- **Frontend:** Vitest + TypeScript / Vite build + npm audit.
- **Navegador:** Playwright + API aislada + modelo doble.
- **Contenedor:** Docker build + semilla léxica + smoke con SHA.

needs: los cinco deben aprobar

- CI aprobado (continúa el flujo).
- Si falla: corregir en la rama y repetir CI.

Tests habituales sin claves ni consumo de modelos.

3 Despliegue automatizado después del merge

Solo main · push o ejecución manual → Guardar la misma imagen Docker que pasó el smoke. → Autenticación GitHub OIDC + WIF. → Publicar imagen en Artifact Registry, etiqueta SHA. → Desplegar en Cloud Run. → Verificar SHA + corpus + rutas retiradas + SSE. → 35 casos contra Cloud Run y Vercel. → Resultado y artefactos de evidencia. → Job rojo: investigar y corregir; sin rollback automático.

Incluye tres consultas generativas reales: directas y por proxy SSE.

Vercel publica el frontend desde Git; **proxy /api** hacia Cloud Run.

Claves de modelos y **ADMIN_PASSWORD:** Secret Manager → cuenta de ejecución de Cloud Run; nunca en frontend o imagen.

Automatizaciones y límites

- **Evaluación real completa:** manual y separada de los tests habituales. — Preparado; verificar merge y ejecución.
- El smoke se ejecuta al construir y después de desplegar. — Preparado; verificar merge y ejecución.
- **Preparado en PR #21:** smoke exige incremento de métricas, evaluación final manual sin despliegue y empaquetador con límite 20 MB. — Preparado; verificar merge y ejecución.

CI verde y despliegue aprobado acreditan esa revisión y esas pruebas; no sustituyen revisión humana, rotación de secretos ni persistencia duradera.

Nuestro proceso de QA: pruebas automáticas y revisión manual

Asistente Educativo UNIANDES · evidencia por capa, sin confundir simulación y producción

Dobles / sin proveedor — Entorno simulado, datos temporales y servicios reemplazados.

Proveedor real — Consume API del servicio en entorno controlado.

Revisión humana — Validación con criterio experto, lectura y contraste de evidencias.

QA automatizado

1 Backend · pytest + cobertura

- Contratos HTTP y validación Pydantic; RAG y citas; agente y lista cerrada; confirmación, expiración e idempotencia; sesiones/CSRF; límites y errores.

Umbral de cobertura ≥85 %

Datos temporales, claves vacías y proveedores bloqueados; dobles declarados.

2 Frontend · Vitest / Testing Library

- Componentes, campos de aclaración, checklist y confirmación.
- TypeScript + Vite build: tipos y compilación.

3 Navegador · Playwright

- Flujos de chat y panel admin; prompts, documentos, índice, análisis, contexto y móvil.

API aislada + modelo doble

4 Calidad y seguridad

- Ruff · Gitleaks · detector de credenciales fijas · pip-audit · npm audit.

Gitleaks verde no sustituye revisión del historial ni prueba ausencia universal de secretos.

5 Docker y smoke

- Usuario sin privilegios; arranque; SHA; corpus; herramientas y calendario; rutas legacy 404; SSE completo.

PR #21: comprobar que una consulta incremente métricas.

6 Pruebas públicas y evaluación real

Activación manual, ejecución automatizada

A 35 casos HTTP en Cloud Run y Vercel: validación, inyección, CORS, admin, herramientas y 3 consultas generativas.

B **Evaluación manualmente activada:** golden set de 20 preguntas, comparación léxico/hibrido y 6 checklists del agente.

Llamadas reales consumen API; criterios y resultados se reportan por separado.

Histórico del agente estricto: 5/6; no equivale al 35/35 público.

QA manual y revisión humana

1 Contenido y fuentes

- Revisar OCR, vigencia y aplicabilidad; leer la respuesta frente al artículo citado; aprobar documentos solo tras revisión.

2 Demo pública y usabilidad

- Pregunta con cita y abrir fuente; fuera de tema/abstención; acción demo con confirmar/cancelar; revisar lectura móvil y navegación.

No mostrar claves ni datos personales.

3 Operación y seguridad

- Vevificar URL y SHA; ensayo de reinicio con respuesta citada; comprobar secretos/permisos y revisiones antiguas; revisar falsos positivos por su contexto.

Persistencia duradera y rotación deben acreditarse, no suponerse.

4 Entrega y sustentación

- Instalación cronometrada por otro integrante; aportes y uso de IA; ensayo de 12 min; vídeo de respaldo de 2 min; capturas y observaciones del tribunal.

Actividades por registrar con evidencia del equipo.

Secuencia de verificación

Cambio en rama → Pruebas locales → PR + CI → Imagen probada → Despliegue en main → Smoke + 35 casos públicos → Demo y revisión humana → Guardar evidencia y pendientes

Corregir causa y repetir la prueba afectada; no relajar aserciones.

Un fallo después de desplegar deja job rojo; no implica rollback automático.

Conclusión:

Reportamos fecha, revisión, resultado esperado/obtenido, denominadores, latencia y límites. Automatizado ≠ exactitud semántica garantizada; una prueba omitida no es una prueba aprobada.

Logs y métricas: qué medimos y para qué sirven

Observabilidad del Asistente Educativo UNIANDES

¿De dónde salen los datos?

Ventana de hasta 2000 eventos; actividad reciente de 50.

Consulta / validación / llamada al proveedor → **Registrar** evento técnico con fecha UTC y **request_id** cuando corresponde → **logs/metrics.jsonl** · rotación 5 MB + 3 respaldos → **GET /api/metricas** → **Dashboard /metricas** · Actualisar datos (carga al abrir y mediante botón)

¿Qué medimos?

Métrica	Significado
Consultas	Respuestas finales, validaciones y rechazos.
Intentos LLM	Cada llamada al proveedor, incluidos fallos y reintentos.
Errores de llamada	Intentos con categoría de fallo.
Respaldo exitoso	Consultas resueltas por el proveedor alternativo.
Latencia media y p95	Duración registrada de consultas en la ventana.
Tokens	Entrada, salida y total reportados por los SDK.
Distribuciones	Tipo de respuesta, modelo llamado y categoría de error.

Una consulta puede provocar varios intentos LLM.

p95 de consultas ≠ prueba de carga ni disponibilidad 24 h.

¿Para qué nos sirve?

Diagnosticar: autenticación, cuota, timeout, modelo no disponible y conexión.

Observar estabilidad: cuántos intentos fallan y cuándo funciona el respaldo.

Comprender rendimiento: latencia de las consultas retenidas.

Revisar consumo: tokens reportados por modelo, sin inventar costos.

Ejemplo conceptual: Consulta → falla Gemini → responde OpenAI: varios intentos, una consulta, respaldo activado.

Eventos y trazabilidad

El contador Intentos LLM y sus tokens corresponde a generación; los eventos embedding no se suman allí como llamadas generativas.

consulta: estado, status y duración.

llm: modelo, intento, fallback, error, duración y tokens si se reportan.

embedding: modelo y operación de indexación/consulta cuando se registra.

request_id: relaciona consulta y llamadas del flujo cuando está disponible.

Privacidad y límites

Allowlist: las métricas no guardan preguntas, respuestas, contexto académico, claves ni mensajes crudos del SDK.

Endpoint público: solo metadatos técnicos; los experimentos administrativos sí pueden guardar textos bajo autenticación.

null = tokens no reportados; no equivale a cero ni permite deducir costo.

Archivo local por instancia: puede reiniciarse o perderse al recrear/desplegar; no es histórico global duradero.

Contadores vacíos no prueban ausencia de uso: verificar revisión, permisos de escritura y reinicios.

Medimos comportamiento técnico; la calidad normativa y el respaldo de cada afirmación requieren evaluación y revisión humana.

UNIANDES
Universidad de los Andes

Asistente Educativo UNIANDES
API · Catálogo de Operaciones de Negocio

Nuestros endpoints: qué realiza cada uno

FastAPI · 31 operaciones agrupadas por función

1. Estado, fuentes y métricas · público

PÚBLICO

#	MÉTODO	RUTA (EXACTA)	DESCRIPCIÓN
1	GET	/health	Estado del proceso y revisión SHA declarada.
2	GET	/ready	Verifica corpus elegible e índice actualizado; no prueba disponibilidad del modelo.
3	GET	/api/sources/{doc_id}/{revision}	Descarga la fuente de la revisión vigente y aprobada.
4	GET	/api/metricas	Resumen y eventos técnicos; sin preguntas ni claves.

2. Chat y agente · público

PÚBLICO

#	MÉTODO	RUTA (EXACTA)	DESCRIPCIÓN
5	POST	/api/chat	Orientación JSON con citas, aclaración, abstención, rechazo o confirmación.
6	POST	/api/agente	Alias del mismo flujo de chat con agente controlado.
7	POST	/api/chat/stream	Mismo flujo; deuvelve eventos SSE token y done tras generar la respuesta.

Nota: Comparten límite de consultas. Con evidencia pueden llamar al modelo; rechazo local o abstención sin evidencia no requieren generación.

3. Herramientas y confirmación

CONFIRMACIÓN

#	MÉTODO	RUTA (EXACTA)	DESCRIPCIÓN
8	GET	/api/herramientas	Esquemas, lista cerrada, límite de vueltas y acciones que requieren confirmación.
9	POST	/api/herramientas/calcular-plazo	Simula días hábiles, sin feriados ni plazo institucional.
10	POST	/api/herramientas/consultar-requisitos	Recupera pasajes pertinentes a una modalidad.
11	POST	/api/herramientas/generar-checklist	Lista orientativa citada, pendiente de verificar.
12	POST	/api/agente/confirmar/{action_id}	Aprueba o cancela un borrador demo con token de capacidad.
13	POST	/api/herramientas/agente/confirmar/{action_id}	Alias de la misma confirmación.

Nota: Las dos confirmaciones exigen token, tienen caducidad e idempotencia; no envían solicitudes a la universidad.

4. Administración: acceso, prompts y configuración

SESIÓN ADMINISTRATIVA

#	MÉTODO	RUTA (EXACTA)	DESCRIPCIÓN
14	POST	/api/admin/login	Valida credenciales y crea cookie de sesión.
15	GET	/api/admin/session	Comprueba sesión y devuelve usuario.
16	POST	/api/admin/logout	Invalida sesión y elimina cookie.
17	GET	/api/admin/bootstrap	Carga prompts, documentos, configuración y estado de modelos sin revelar claves.
18	POST	/api/admin/prompts	Crea una nueva versión de prompt.
19	POST	/api/admin/prompts/{version}	Edita una versión con control de revisión.
20	POST	/api/admin/config	Guarda prompt activo, estrategia RAG y parámetros.

Nota: Login requiere guard de mutación; el resto exige sesión. Las mutaciones usan **X-Admin-Request** y validación de origen.

5. Administración: documentos, RAG y análisis

SESIÓN ADMINISTRATIVA

#	MÉTODO	RUTA (EXACTA)	DESCRIPCIÓN
21	POST	/api/admin/documents	Upload de archivo y metadatos; guarda documento y revisión.
22	POST	/api/admin/documents/{doc_id}	Edita metadatos, vigencia, aprobación y habilitación con control de revisión.
23	POST	/api/admin/documents/{doc_id}/replace	Reemplaza original; nueva revisión queda sin aprobar.
24	GET	/api/admin/documents/{doc_id}/file	Descarga original actual o revisión histórica disponible.
25	GET	/api/admin/history	Consulta auditoría del recurso indicado.
26	GET	/api/admin/rag/chunks	Lista fragmentos de un documento.
27	POST	/api/admin/rag/index/{doc_id}	Indexa la revisión; embeddings solo si modo híbrido.
28	POST	/api/admin/rag/search	Busca evidencia y devuelve traza; no genera respuesta de chat.
29	POST	/api/admin/analysis	Inicia comparación de prompts en segundo plano (202), proveedor elegido sin fallback.
30	GET	/api/admin/analysis	Lista experimentos administrativos recientes.
31	GET	/api/admin/analysis/{job_id}	Recupera resultados, evidencia y trazas del experimento.

Nota: Todo este panel exige sesión; las mutaciones además requieren guard. Comparar no activa un ganador y no usa juez IA adicional.

Lectura rápida:

- **Público no significa sin validación.**
- **401** sesión ausente · **403** guard/origen · **409** conflicto · **410** confirmación vencida · **422** entrada inválida · **429** límite · **503** servicio no disponible.

Son códigos según endpoint, no todos para cada ruta.

Documentación automática: **/docs** · contrato: **/openapi.json**. **/admin** y **/metricas** son rutas de la interfaz. **/admin/login** y **/admin/recargar-reglamentos** son rutas legacy retiradas (404).

Una consulta del chatbot: del frontend a la respuesta

Validación · RAG · agente · modelos · fallback · SSE

1 Frontend y entrada

Usuario: pregunta + contexto opcional + modelo preferido (Gemini / OpenAI)

React envía POST /api/chat/stream; /api/chat y /api/agente comparten flujo

Presupuesto compartido: 30 POST / 60 s / proceso

¿Presupuesto agotado?

Sí → **HTTP 429 + Retry-After** → frontend muestra espera y conserva la consulta

No → Validar **ConsultaChat** (Pydantic)

¿Válida?

No → **HTTP 422** → corregir campos

Sí

El presupuesto se aplica antes de validar el cuerpo.

2 Seguridad y recuperación

¿Marcador de inyección conocido?

Sí → **Rechazo local,** sin RAG ni generación (HTTP 200, estado: rechazo)

No → **Preparar búsqueda y recuperar fuentes** aprobadas, habilitadas, vigentes e indexadas.

Léxico: FTSS / BM25

Híbrido opcional: embedding de consulta + coseno + RRF

Error de embeddings: Aviso y respaldo BM25; no fallback automático de embeddings a otro proveedor.

¿Fuera del dominio y sin evidencia?

Sí → **Rechazo local** (HTTP 200, estado: rechazo)

No → ¿En dominio pero sin evidencia suficiente?

Sí → **Abstención local** (HTTP 200, estado: abstencion)

No → **Con evidencia:** prompt activo + reglas + contexto y fragmentos

El rechazo y la abstención locales retornan HTTP 200 con su estado, no son errores HTTP.

3 Bucle del agente · máximo 4 vueltas

Herramientas posibles: buscar_fragmentos · consultar_requisitos · generar_checklist · calcular_plazo · registrar_solicitud_tutor

Llamar ClienteLLM con modelo preferido

Interpretar JSON del modelo

¿Pide herramienta o respuesta final?

Herramienta → Validar lista cerrada, esquema y datos declarados.

¿Solicitud válida?

No → Agregar error controlado al contexto → siguiente vuelta.

Sí → ¿Qué herramienta?

registrar_solicitud_tutor: Validar petición explicita y datos → proponer borrador demo → salir con estado confirmación. (No ejecutar escritura definitiva aquí).

Herramienta de lectura/cálculo (p. ej., buscar_fragmentos, consultar_requisitos, generar_checklist, calcular_plazo): Ejecutar Python, recuperar evidencia y agregar resultado al contexto → siguiente vuelta.

Respuesta final → **Salir a validación de respuesta** (Sección 4).

Contador de vueltas (inicia en 1)

¿Tope (4 vueltas) alcanzado?

No → vuelve a Llamar ClienteLLM

Sí → **Abstención por límite de pasos;** no ejecutar herramientas adicionales.

Siguiente vuelta (**vuelta + 1, máximo 4**)

Reintento de proveedor ≠ vuelta del agente ≠ nueva consulta del usuario. | No confundir el reintento técnico del proveedor (dentro de una llamada) con otra vuelta del agente ni con un reintento manual del usuario.

Dentro de cada llamada al modelo

Elegir proveedor preferido si tiene clave.

Intentar generación: timeout configurado 15 s por llamada; LLM_RETRIES=0 por defecto.

Éxito (no vacio): Devolver salida y uso reportado al agente.

Fallo: Registrar categoria segura: autenticación, cuota, timeout, modelo no disponible, proveedor o conexión.

¿Hay otro proveedor configurado y fallback permitido?

Sí → **Intentar proveedor alternativo:** Gemini ↔ OpenAI, según preferencia.

Éxito alternativo: Continuar con fallback_activado=true.

No → **Sin proveedores o ambos fallan:** HTTP 503 con mensaje seguro → frontend conserva datos y permite reintentar.

SDK sin reintentos internos adicionales. Si se configura LLM_RETRIES > 0, solo se reintentan cuota/timeout/proveedor, con espera exponencial y jitter; después se prueba el respaldo.

Generación: Gemini gemini-3.5-flash-lite / OpenAI gpt-4.1-mini (configurables).

4 Validar y devolver

Salida final del agente (respuesta, aclaración o abstención).

Validar **BorradorRAG:** JSON, estado, IDs citados y vigencia.

¿Válido?

Sí → **respuesta:** texto + fuentes verificables. | **aclaracion:** campos académicos permitidos que faltan. | **abstencion / rechazo:** explicar límite.

Usuario completa datos → NUEVA consulta con pregunta y contexto; no memoria conversacional persistente.

No → **Abstención segura:**
- JSON inválido.
- Cita inexistente.
- Respuesta normativa sin citas.
- Evidencia cambiada.
- Aclaración sin campos pendientes.

Confirmación humana (desde registrar_solicitud_tutor) — Usuario revisa argumentos y aprueba/cancela mediante endpoint confirmar + token. → Guardar borrador demo local o cancelar; TTL e idempotencia; sin modelo para decisión.

Salida al frontend (éxitos)

Contrato **RespuestaAsistente** → SSE: eventos token y done → React muestra texto, fuentes, traza, campos o confirmación.

El backend completa la generación antes de emitir SSE: no son tokens transmitidos en vivo desde el SDK.

HTTP 422 / 429 / 503: se producen antes de iniciar SSE y se manejan como error JSON.

Tiempos y observabilidad

- 15 s es timeout por llamada, no duración máxima de toda la consulta. RAG + varias vueltas + fallback pueden sumar más tiempo. La latencia se mide, no se garantiza.
- Registrar métricas técnicas: consulta, intentos LLM, modelo, fallback, duración, tokens y error; sin preguntas ni claves en métricas públicas.

Configuración del asistente: secretos, variables y parámetros

Asistente Educativo UNIANDES · nombres reales y valores de referencia

⚠ **Los valores indicados son defaults del código; el despliegue y la configuración guardada pueden cambiarlos.**

1 Privadas: nunca en frontend ni Git

Nombre	Valor	Función
GEMINI_API_KEY	🔒 **secreto**	credencial del proveedor Gemini
OPENAI_API_KEY	🔒 **secreto**	credencial del proveedor OpenAI
ADMIN_PASSWORD	🔒 **secreto**	contraseña del panel administrativo

Producción: Secret Manager → variables de Cloud Run; cuenta de ejecución con permiso de acceso.

Los Secrets de GitHub Actions para evaluación se configuran aparte; no heredan los de Cloud Run.

GCP_SA_KEY: secreto opcional de despliegue si NO se usa WIF; no necesario con WIF.

2 Backend: entorno, sesión y respuesta

Nombre	Función
APP_NAME	nombre de la aplicación
APP_ENV	development / production
APP_REVISION	SHA declarado del build
CORS_ORIGINS	orígenes autorizados, sin comodín
ADMIN_USERNAME	usuario administrativo (default admin; no es contraseña)
ADMIN_COOKIE_SECURE	false local; true en HTTPS producción
ADMIN_SESSION_MINUTES	60 min
LLM_TIMEOUT_SECONDS	15 s por llamada
LLM_RETRIES	0 por defecto
LLM_MAX_INPUT_CHARS	2000 caracteres
RAG_DATA_DIR	ubicación del estado SQLite y originales
PORT	puerto usado por Uvicorn en el contenedor (8080 por defecto)

15 s no es duración total **garantizada**; agente y fallback pueden sumar llamadas.

get_settings usa caché; cambiar variables/.env requiere reiniciar o desplegar revisión.

3 Modelos: nombres públicos, claves privadas

Nombre	Función
AI_MODEL	gemini-3.5-flash-lite; generación Gemini
FALLBACK_MODEL	gpt-4.1-mini: generación OpenAI
embedding_model	gemini-embedding-001 o text-embedding-3-small
modelo_preferido	gemini / openai, enviado por consulta

- **AI_PROVIDER** y **FALLBACK_PROVIDER** están declaradas, pero el orden real de ClienteLLM lo decide modelo_preferido y las claves disponibles.
- **AI_API_KEY** aparece en el ejemplo .env, pero NO se lee como clave en Settings.
- Elegir modelo de chat no cambia embeddings.

4 Parámetros de negocio · config/parametros,yaml

Parámetro (valor de referencia)	Función
max_iteraciones: 4	vueltas del agente
max_dias_habiles: 730	tope del calendario simulado
chat_rate_limit: 30 solicitudes / 60 s / proceso	límite de solicitudes del chat
embedding_texts_per_minute: 80	presupuesto Gemini; no cuota universal
embedding_retries: 2	reintentos propios de embeddings
rrf_k: 60	fusión de rankings
checklist: max_items 20 · min_caracteres 25 · max_evidencias 40	restricciones del checklist (items, caracteres y evidencias)

5 Configuración RAG y criterios de evaluación

RAG (recuperación y fragmentación)

Parámetro	Función
active_prompt: 3	prompt activo del RAG
chunk_size: 1400 · overlap: 160	tamaño de fragmento y solapamiento
top_k: 4	número de pasajes recuperados
strategy: lexical / hybrid	estrategia de recuperación
min_lexical_coverage: 0.60	cobertura mínima léxica (0.60)
min_similarity: Gemini 0.65 / OpenAI 0.55	umbral de similitud de referencia (Gemini 0.65 / OpenAI 0.55)

Defaults RAG provienen del archivo; el panel guarda configuración en SQLite. La imagen Docker siembra explícitamente lexical.

Evaluación (criterios de aceptación)

Parámetro	Función
min_casos: 20 · min_negativos: 4	tamaño mínimo del conjunto de evaluación
acierto_top_k ≥0.70 · silencio ≥0.90	umbrales de acierto en top-k y de silencio
acierto_cita ≥0.80 · abstencion_o_rechazo =1.00	umbral de acierto de cita y de abstención o rechazo

Son criterios de aceptación, no resultados obtenidos.

6 Públicas: frontend y despliegue

Frontend (variables del build de Vite)

Nombre	Función
VITE_API_URL	URL pública del backend; incluida en el build
VITE_ADMIN_API_TARGET	destino del proxy de Vite en desarrollo

CI/CD (despliegue en Google Cloud)

Nombre	Función
GCP_PROJECT_ID	ID de proyecto
GCP_AR_REPOSITORY	repositorio Docker
GCP_WORKLOAD_IDENTITY_PROVIDER	recurso del proveedor WIF
GCP_SERVICE_ACCOUNT	cuenta de despliegue
GCP_REGION=us-central1 · GCP_SERVICE=titulacion-rag	definidos en workflow

Identificadores ≠ credenciales. Toda variable VITE_* debe considerarse pública; nunca poner claves allí.

No versionar .env con secretos. | **RAG_DATA_DIR no crea persistencia duradera en Cloud Run.** | **Cambiar modelo de embeddings o fragmentación puede exigir reindexar; no mezclar vectores de modelos distintos.**